



Cockroach Browser: A Local-First Browser Runtime for AI Agents

Author: Ajnas NB

Paper version: 1.0

Implementation: Cockroach Browser 0.2.0

Date: July 2026

Software license: AGPL-3.0-or-later

Document license: Creative Commons Attribution 4.0 International

Status: Implementation-backed technical white paper for Cockroach Browser 0.2.0. The paper has not undergone independent peer review.

Abstract

AI agents can use a browser to inspect dynamic applications, fill forms, download files, capture evidence, and complete operational workflows. A conventional automation process, however, often inherits more authority than the task requires: ambient browser profiles, persistent cookies, arbitrary origins, unrestricted JavaScript, local files, broad network reach, or an unauthenticated remote control port.

Cockroach Browser is a local-first TypeScript and Chromium runtime that separates browser capability from ambient machine authority. A host creates an explicit session with a purpose, allowed origins, allowed actions, allowed effects, and finite budgets. The runtime then provides semantic page references, browser interactions, screenshots, PDFs, traces, network observations, audits, and hash-linked receipts within that session. Login, consent, CAPTCHA, and access challenges pause for human handling or a separately authorized resolver; they are not bypassed.

Version 0.2.0 exposes an embedded SDK, an authenticated loopback daemon, a typed client, a command-line interface, an observation-first MCP server, Docker deployment, a local dashboard, per-user service definitions, and adapters for Maqam, Qarinah, Cockroach Crawler, and ProductLoop OS. Its source-derived capability registry contains 80 entries: 73 implemented surfaces, 6 host-backed adapters, and 1 explicitly planned capability.

1. Problem

Browser automation creates a difficult authority boundary. A browser can see and change real systems, but a model should not silently decide which profiles, credentials, origins, local resources, or network destinations become available.

Three common designs are incomplete:

1. A general desktop browser gives an agent broad ambient authority.
2. A remote browser service centralizes credentials and evidence in another operator's infrastructure.
3. A narrow scraping interface cannot complete stateful, interactive workflows.

Cockroach Browser treats browser access as a session-scoped capability. The browser remains useful, but the host owns the authority envelope.

2. Design goals

The runtime is designed around seven goals:

1. Real Chromium rendering and interaction.
2. Explicit authority for every session.
3. Finite resource and evidence budgets.
4. Stable observations through snapshot-scoped semantic references.
5. Paired visual and structural evidence.
6. Authenticated local and remote control surfaces.
7. Separate governance, memory, crawling, and orchestration ledgers connected by stable identifiers.

The system is not designed to bypass access controls, CAPTCHAs, paywalls, rate limits, robots policies, or site authorization.

3. System model

The main actors are:

- **Deployment owner:** installs the runtime and owns configuration, secrets, storage, and network exposure.
- **Host application:** creates sessions, supplies explicit authority, and decides which surfaces an agent may reach.
- **Agent:** consumes observations and proposes or requests allowed actions.
- **Browser runtime:** enforces the session policy, drives Chromium, and records evidence.
- **Human reviewer:** handles login, consent, challenges, and consequential approvals when required.
- **Maqam:** evaluates policy, binds exact approvals, dispatches governed actions, and records governance receipts.
- **Qarinah:** preserves cited project memory without becoming browser authority.
- **Cockroach Crawler:** maps and extracts bounded public web content before selected pages require a browser.

4. Architecture

Cockroach Browser has four layers:

4.1 Runtime

BrowserRuntime owns Chromium lifecycle, authorized sessions, action classification, policy checks, budgets, evidence records, receipts, queues, and integrations.

4.2 Control surfaces

The same runtime can be reached through:

- an embedded TypeScript SDK
- an authenticated HTTP daemon
- a typed daemon client
- a command-line interface
- an observation-first MCP server
- a local operator dashboard

4.3 Evidence

Evidence includes semantic snapshots, screenshots, paired captures, PDFs, Playwright traces, HAR files, video, console records, network metadata, downloads, audit results, annotations, and action receipts. Records are content-addressed and linked to the session that produced them.

4.4 Integrations

Adapters connect browser evidence to Maqam policy and approvals, Qarinah project memory, Cockroach Crawler handoffs, and ProductLoop capability descriptions. The adapters do not collapse these systems into one authority or one ledger.

5. Authorized sessions

Every session states:

- purpose
- start URL
- allowed origins
- allowed actions
- allowed effects
- actions that require approval
- action, duration, tab, and evidence budgets

The runtime requires at least one explicit allowed origin. Private and loopback destinations are denied unless the deployment owner opts in. Redirects are revalidated. Host-controlled executable paths, CDP endpoints, proxy settings, headers, profile secrets, and remote listeners remain unavailable unless the host explicitly exposes them.

Named profiles are isolated. Storage state can be imported or exported only through explicit host operations. The runtime does not discover ambient browser profiles or cookies.

6. Semantic observations and interaction

A snapshot converts the current page into bounded text, structure, metadata, and semantic references. Each reference belongs to the observed page revision. A later action cannot silently reuse a stale reference after the page changes.

Implemented interactions include navigation, tabs, popups, click and double-click, forms, keyboard and mouse input, drag and drop, bounded scroll, waits, same-origin frames, open Shadow DOM, dialogs, uploads, downloads, extraction, screenshots, PDFs, tracing, and policy-gated JavaScript.

Actions are classified by effect and risk before dispatch. A session may permit read observations while requiring an exact Maqam approval for a write, upload, download, dialog acceptance, or arbitrary evaluation.

7. Evidence and receipts

Browser output is useful only when a later reviewer can connect it to the session, input, policy, and observed result.

Each action receipt records:

- canonical action material and digest
- policy digest

- URL before and after
- result status
- evidence identifiers
- previous receipt hash

The linked receipt chain exposes missing, reordered, or modified records. Evidence verification recomputes content digests without replaying the browser session.

Paired capture records a screenshot and semantic snapshot from the same reviewed page state. Temporary annotations can label selected references without changing application data.

8. Challenge handling

The runtime detects login, consent, CAPTCHA, and access challenges. Detection pauses the automated path and records a challenge state. A human can complete the required step in a headed session, or an explicitly authorized resolver can handle a supported challenge under host policy.

The runtime does not claim CAPTCHA bypass, stealth evasion, credential discovery, access-control bypass, or authorization circumvention.

9. Network observations

The runtime can record bounded network metadata and export redacted observations. Session policy controls permitted origins and routing behavior.

Network interception is limited to policy-bounded request blocking or bounded static fulfillment. Webhook delivery uses public HTTPS destinations, revalidates DNS, rejects private or mixed-address results, preserves TLS hostname verification, and does not follow redirects.

10. MCP

The native stdio MCP server is observation-first. It exposes:

- browser_capabilities
- browser_health
- browser_sessions
- browser_snapshot
- browser_audit
- browser_capture
- browser_network
- browser_propose_action

browser_propose_action returns canonical action material and an input digest. It does not execute the action. Session creation, profile import, login, secret resolution, remote binding, and mutation authority stay with the host.

11. Maqam governance

For consequential actions, Cockroach Browser can produce a proposal for Maqam. Maqam evaluates policy, binds an approval to the exact operation, dispatches through its registered driver, rejects replay or changed input, and records a governance receipt.

Browser evidence proves what Chromium observed or executed. Maqam proves the policy, approval, and dispatch path. Stable identifiers connect the records without treating them as one database.

12. Qarinah memory

The Qarinah adapter records bounded browser outcome material such as canonical input and output digests, evidence identifiers, receipt hashes, and cited descriptive metadata. It filters cookies, storage values, form values, and secrets.

Qarinah does not dispatch browser actions. It lets later agents retrieve cited project memory without replaying complete project histories or treating a generated summary as the source of truth.

13. Cockroach Crawler handoff

Cockroach Crawler handles bounded static HTTP crawling, mapping, feeds, public-source adapters, structured extraction, and document discovery. Cockroach Browser handles stateful rendering, interactive pages, authenticated user-approved sessions, and browser evidence.

A typical workflow maps a site with the crawler, ranks candidate URLs, and opens only selected pages in the browser. This preserves browser budgets and makes the reason for each rendered session explicit.

14. Deployment

Cockroach Browser supports:

- project-local npm installation
- global per-user CLI installation
- one-command bootstrap and health checks
- shell completion generation
- current-user autostart definitions
- embedded SDK use
- authenticated loopback daemon
- authenticated remote workers with explicit TLS and remote mode
- Docker

Global installation makes the CLI available to the current computer account. It does not grant browser profiles, cookies, origins, credentials, or machine resources.

15. Security model

The default daemon:

- binds to loopback
- requires a cryptographically strong bearer token
- stores generated tokens with restrictive permissions
- rejects non-loopback binding unless remote mode and TLS are explicit
- disables raw HTTP action dispatch by default
- enforces origin and private-network policy
- enforces finite action, duration, tab, upload, download, history, and evidence budgets
- pauses on access challenges
- records content-addressed evidence and hash-linked receipts

The supported security line is 0.2.x. Private vulnerability reporting is the required disclosure channel.

16. Capability status

The source-derived registry for 0.2.0 contains:

- 80 total capability entries
- 73 available in the package
- 6 adapter-backed integrations
- 1 planned team synchronization capability

The registry covers sessions, profiles, state checkpoints, tabs, interaction, frames, dialogs, history, JavaScript, files, screenshots, paired evidence, PDFs, traces, HAR, video, console, network records, receipts, extraction, audits, challenges, origin policy, private-network policy, effects, budgets, routes, approvals, secrets, CLI, completions, services, bootstrap, SDK, HTTP, MCP, Docker, dashboard, remote workers, local jobs, health checks, Maqam, Qarinah, Cockroach Crawler, ProductLoop, webhooks, and team synchronization status.

17. Verification

The 0.2.0 release candidate is verified through:

- Node.js 22, 24, and 26 CI
- real Chromium boundary tests
- packed npm consumer tests
- hardened Docker smoke tests
- dependency review
- CodeQL
- npm audit
- generated documentation checks

- immutable npm artifact identity checks in the release workflow

The tests establish behavior for the committed fixtures and configured environments. They are not a claim that every website, browser challenge, operating system policy, or third-party integration has been independently certified.

18. Limitations

Cockroach Browser does not:

- bypass access controls or CAPTCHAs
- grant agents ambient machine authority
- discover arbitrary cookies or profiles
- make remote unauthenticated binding safe
- make browser evidence equivalent to governance approval
- guarantee that a website permits automation
- replace a distributed queue or multi-host consensus system
- claim independent security certification

19. Reproducibility

The source repository contains the implementation, capability registry, manuals, tests, Docker assets, release workflow, and generated website:

<https://github.com/AjnasNB/cockroach-browser>

The public documentation is:

<https://cockroachbrowser.com>

The npm package is:

<https://www.npmjs.com/package/cockroach-browser>

To reproduce the main verification:

```
npm ci
npm run check
COCKROACH_BROWSER_E2E=1 npm test
node scripts/test-packed-consumer.mjs
```

20. Conclusion

Cockroach Browser gives an AI agent a real browser without silently transferring the whole machine's authority. The central design choice is not to remove browser capability, but to bind it to explicit sessions, finite budgets, evidence, and host-owned control.

This makes browser automation composable with crawling, memory, policy, approval, and workflow systems while preserving the distinct proof each layer provides.

Acknowledgements

The author thanks Shahin Ahammed for contributions to product direction, use-case definition, positioning, and review of the manuscript.

References

1. Cockroach Browser source and documentation, version 0.2.0.
2. Model Context Protocol documentation.
3. Playwright documentation.
4. Maqam source and documentation.
5. Qarinah technical white paper and implementation.
6. Cockroach Crawler source and documentation.